

Java Identifiers and Data Types

```
...or object to mirror  
mirror_mod.mirror_object =  
operation == "MIRROR_X":  
    mirror_mod.use_x = True  
    mirror_mod.use_y = False  
    mirror_mod.use_z = False  
operation == "MIRROR_Y":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = True  
    mirror_mod.use_z = False  
operation == "MIRROR_Z":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = False  
    mirror_mod.use_z = True
```

```
...selection at the end -add  
mirror_ob.select= 1  
mirror_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier_ob))  
mirror_ob.select = 0  
bpy.context.selected_object  
data.objects[one.name].select  
print("please select exactly")
```

```
-- OPERATOR CLASSES -----
```

```
types.Operator):  
    X mirror to the selected  
    object.mirror_mirror_x"  
    error X"
```

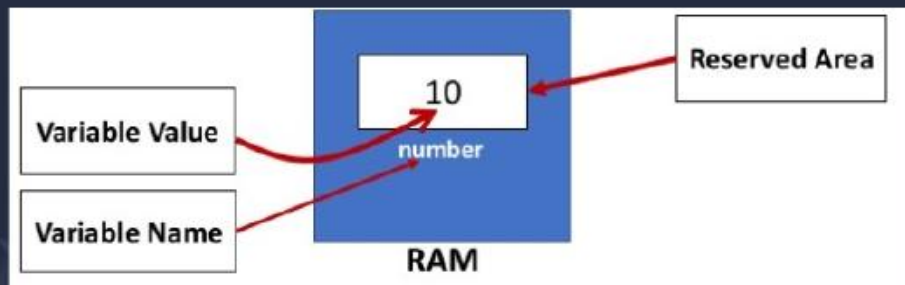
Objectives

- Understand how variables are created in Java.
- Discuss the importance of Identifiers in Java
- Define data type and its classifications.
- Define type casting and its application in Java.



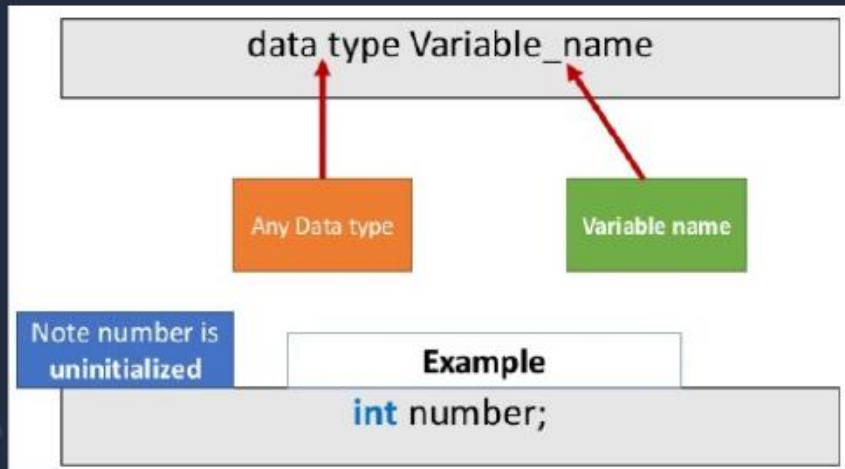
Review on Variables:

- **Variable** is a name of memory location and data type specifies the size and the type of value it can store.
- A variable provides us with named storage that our programs can manipulate.



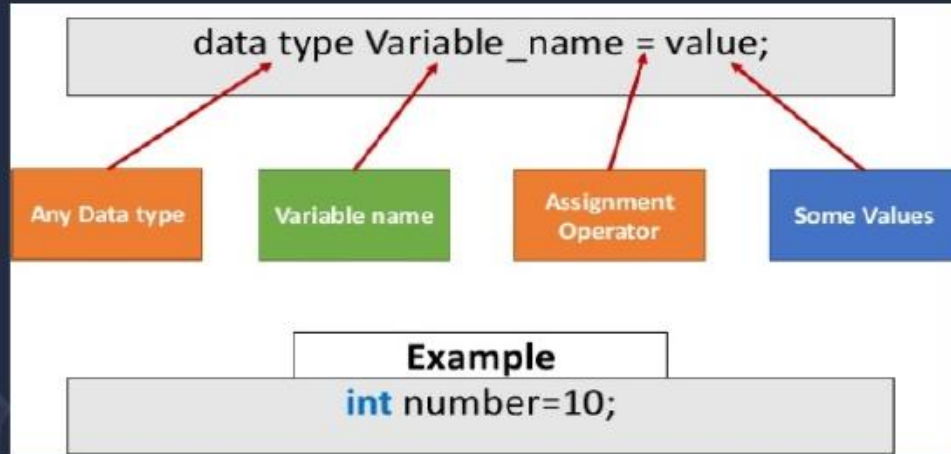
Review on Variables:

- **Variable declaration**



Review on Variables:

- **Variable Declaration and Initialization**



Java Identifiers

- All Java **variables** must be **identified** with **unique names**.
- These unique names are called **identifiers**.
- Identifiers can be short names (like `x` and `y`) or more descriptive names (`age`, `sum`, `totalVolume`).
- **Note:** It is recommended to use descriptive names in order to create understandable and maintainable code.



Java Identifiers: Naming rules

All java components require names. Names are used for classes, methods, variables and interfaces. These names are called identifier. Identifiers follow naming rules:

1. All identifiers must start with either a letter or a currency character \$ or an underscore _
2. After the first character, the identifier can have any combination of characters
3. A java keyword can't be used as an identifier
4. It is case sensitive
5. Names cannot contain whitespace

Java Identifiers

```
// Good
```

```
int minutesPerHour = 60;
```

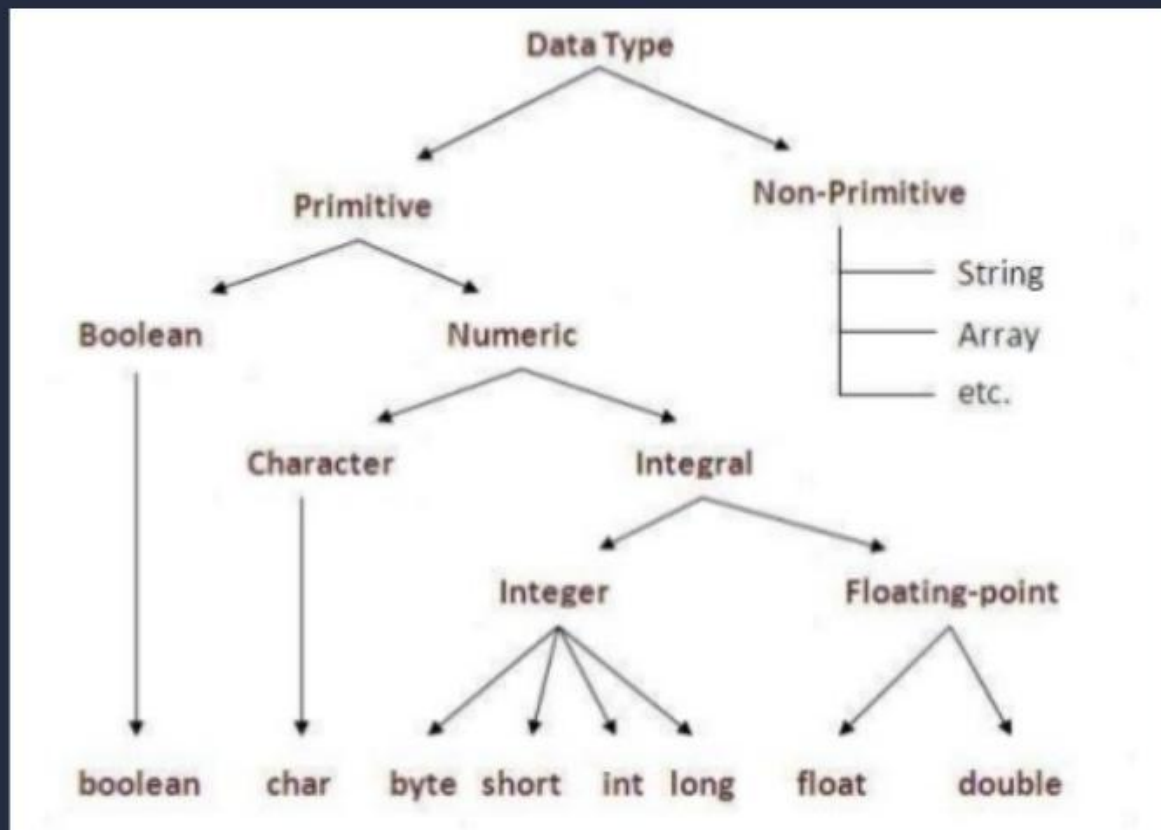
```
// OK, but not so easy to understand what m actually is
```

```
int m = 60;
```



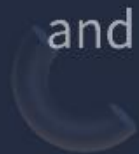
Data Types in Java

- A **data type** is a classification that dictates what a variable or object can hold.



Primitive Data Types

- There are **eight** primitive data types supported by Java.
- Primitives are predefined by the programming language and named by a keyword.
- These are the data types that allows us to store only one value.
- A **primitive data type** specifies the size and type of variable values, and it has no additional methods.



Primitive Data Types

Data Type	Size	Description
byte	1 byte	Stores whole numbers from -128 to 127
short	2 bytes	Stores whole numbers from -32,768 to 32,767
int	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
double	8 bytes	Stores fractional numbers. Sufficient for storing 15 decimal digits
boolean	1 bit	Stores true or false values
char	2 bytes	Stores a single character/letter or ASCII values

Example

• Example

- `int a;` `// valid`
- `a=10;` `// valid`
- `a=10, 20, 30;` `// invalid`

These are
Comment

Data Type

Variable
Name

Primitive Number Types

Primitive number types are divided into two groups:

- **Integer types** stores whole numbers, positive or negative (such as 123 or -456), without decimals. Valid types are `byte`, `short`, `int` and `long`. Which type you should use, depends on the numeric value.
- **Floating point types** represents numbers with a fractional part, containing one or more decimals. There are two types: `float` and `double`.

Integer Types

- **Byte**

The byte data type can store whole numbers from -128 to 127.

This can be used instead of int or other integer types to save memory when you are certain that the value will be within -128 and 127:

```
byte firstnum=100;  
System.out.println(firstnum);
```

Integer Types

- **Short**

- The short data type can store whole numbers from -32768 to 32767:

```
short firstnum=1000;  
System.out.println(firstnum);
```

Integer Types

- **Int**

- The int data type can store whole numbers from -2147483648 to 2147483647.
- In general, and in our tutorial, the int data type is the preferred data type when we create variables with a numeric value.

```
int firstnum=100000;  
System.out.println(firstnum);
```

Integer Types

- **Long**

- The long data type can store whole numbers from -9223372036854775808 to 9223372036854775807. This is used when int is not large enough to store the value.
- Note that you should end the value with an "L":

```
long firstnum=4000000000L;  
System.out.println(firstnum);
```

Floating-Point Types

- You should use a floating-point type whenever you need a number with a decimal, such as 9.99 or 3.14515.

- **Float**

- The float data type can store fractional numbers from $3.4e-038$ to $3.4e+038$. Note that you should end the value with an "f":

```
float firstnum=5.56f;  
System.out.println(firstnum);
```

Floating-Point Types

- **Double**

- The double data type can store fractional numbers from $1.7e-308$ to $1.7e+308$.
- Note that you should end the value with a "d":

```
double firstnum=5.456d;  
System.out.println(firstnum);
```

Float or Double?

- The **precision** of a floating-point value indicates how many digits the value can have after the decimal point.
- The precision of float is only six or seven decimal digits, while double variables have a precision of about 15 digits.
- Therefore it is safer to use double for most calculations.



Scientific Numbers

- A floating-point number can also be a **scientific number** with an "e" to indicate the power of 10:

```
float firstnum=23e4f;  
double secondnum=12e5d;  
System.out.println(firstnum);  
System.out.println(secondnum);
```

Booleans

- A **boolean** data type is declared with the **boolean** keyword and can only take the values `true` or `false`:

```
boolean isProgramming=true;  
boolean isdifficult=false;  
System.out.println(isProgramming);  
System.out.println(isdifficult);
```



Characters

- The **char** data type is used to store a **single character**.
- The character must be surrounded by single quotes, like 'A' or 'c'.

```
char grade='A';  
System.out.println(grade);
```

Characters

- Alternatively, you can use **ASCII** values to display certain characters:

```
char a=65, b=66, c=67;  
System.out.println(a);  
System.out.println(b);  
System.out.println(c);
```

ASCII

- **ASCII** stands for the "American Standard Code for Information Interchange".
- It was designed in the early 60's, as a standard character set for computers and electronic devices.
- ASCII is a 7-bit character set containing 128 characters.
- **The ASCII control characters (range 00-31, plus 127) were designed to control hardware devices.**

ASCII Printable Characters

Char	Number	Description
	0 - 31	Control characters (see below)
	32	space
!	33	exclamation mark
"	34	quotation mark
#	35	number sign
\$	36	dollar sign
%	37	percent sign
&	38	ampersand
'	39	apostrophe
(	40	left parenthesis
)	41	right parenthesis
*	42	asterisk
+	43	plus sign
,	44	comma

-	45	hyphen
.	46	period
/	47	slash
0	48	digit 0
1	49	digit 1
2	50	digit 2
3	51	digit 3
4	52	digit 4
5	53	digit 5
6	54	digit 6
7	55	digit 7
8	56	digit 8
9	57	digit 9
:	58	colon
;	59	semicolon
<	60	less-than

=	61	equals-to
>	62	greater-than
?	63	question mark
@	64	at sign
A	65	uppercase A
B	66	uppercase B
C	67	uppercase C
D	68	uppercase D
E	69	uppercase E
F	70	uppercase F
G	71	uppercase G
H	72	uppercase H
I	73	uppercase I
J	74	uppercase J
K	75	uppercase K
L	76	uppercase L

Strings

- The **String** data type is used to store a sequence of characters (text). String values must be surrounded by double quotes:

```
String happy= "I am happy!";  
System.out.println(happy);
```



Non-Primitive Data Types

- **Non-primitive data types** are called **reference types** because they refer to objects.
- The main difference between **primitive** and **non-primitive** data types are:
- Primitive types are predefined (already defined) in Java. Non-primitive types are created by the programmer and is not defined by Java (except for String).
- Non-primitive types can be used to call methods to perform certain operations, while primitive types cannot.

Non-Primitive Data Types

- The main difference between **primitive** and **non-primitive** data types are:
- A primitive type has always a value, while non-primitive types can be null.
- A primitive type starts with a lowercase letter, while non-primitive types starts with an uppercase letter.
- The size of a primitive type depends on the data type, while non-primitive types have all the same size.
- Examples of non-primitive types are Strings, Arrays, Classes, Interface, etc. You will learn more about these in a later chapter.

Keywords in Java

- In java, a **keyword** is a word with a predefined meaning in Java programming syntax.
- These are **reserved words**, and it should not be used in naming classes, method or variables.

Keywords in Java				
abstract	default	if	private	this
assert	do	implements	protected	throw
boolean	double	import	public	throws
break	else	instanceof	return	transient
byte	enum	int	short	try
case	extends	interface	static	void
catch	final	long	strictfp	volatile
char	finally	native	super	while
class	float	new	switch	
continue	for	package	synchronized	

Type Casting in Java

- **Type casting** is when you assign a value of one primitive data type to another type.
- In Java, there are two types of casting:

- **Widening Casting** (automatically) - converting a smaller type to a larger type size

`byte -> short -> char -> int -> long -> float -> double`

- **Narrowing Casting** (manually) - converting a larger type to a smaller size type

`double -> float -> long -> int -> char -> short -> byte`

Type Casting in Java: Widening Casting

- **Widening casting** is done automatically when passing a smaller size type to a larger size type:

```
public class MyClass {  
    public static void main(String[] args) {  
        int myInt = 9;  
        double myDouble = myInt; // Automatic casting: int to double  
  
        System.out.println(myInt);    // Outputs 9  
        System.out.println(myDouble); // Outputs 9.0  
    }  
}
```

Type Casting in Java: Narrowing Casting

- **Narrowing casting** must be done manually by placing the type in parentheses in front of the value:

```
public class MyClass {  
    public static void main(String[] args) {  
        double myDouble = 9.78;  
        int myInt = (int) myDouble; // Manual casting: double to int  
  
        System.out.println(myDouble); // Outputs 9.78  
        System.out.println(myInt);    // Outputs 9  
    }  
}
```